

CL-2005: Database Systems

Lab # 3: Introduction to SQL

Objective

To explore concepts i.e., DDL, DML, DCL and DQL.

Scope:

The student shall know the following:

- Workaround SQL Server.
- SQL Practice.
- Hands-on experience on the above-mentioned concepts.

Discussion:

SQL Basics

2.1. SQL (Structured Query Language)

- **Definition:** SQL is a language used for managing and manipulating relational databases.
 - **Purpose:** It helps in querying, updating, and managing data.
-

SQL Languages:

SQL Language is made up of various types of SQL Commands. The SQL language commands are divided into four main sections:

- **DDL** (Data Definition Language) Commands:
 - DDL commands are used to manipulate the database itself. They deal with creation of database, altering its structure and deleting the database.
- **DML** (Data Manipulation Language) Commands:
 - DML commands are used to insert, update and delete data in the database. As the name suggests, they are used to manipulate the data in database.
 - The DML commands are INSERT, UPDATE and DELETE.
 - The DDL Commands are CREATE, ALTER and DROP.
- **DCL** (Data Control Language) Commands:
 - DCL commands are used to control the access to the database. They are used to give privilege to the users and take away privilege from the users.
 - The DCL Commands are GRANT and REVOKE.
- **DQL** (Data Query Language) Command:
 - Data is accessed from the database by firing queries. DQL command allows us to fire queries to the database and get the data from the database.
 - The DQL command is SELECT.

1. Data Definition Language (DDL):

The various DDL commands are **CREATE**, **ALTER** and **DROP**. Let us learn about each DDL Command one by one with examples.

2.1 CREATE Statement:

The **CREATE** statement in SQL is used to create the database and the database objects like **Tables**, **Views**, etc. Let us see how a database and table are created in SQL.

Creating a Database

- **Command:** CREATE DATABASE databaseName;
- **Purpose:** Initializes a new database for storing tables and data.

Example:

```
CREATE DATABASE RetailStore;
```

Using a Database

- **Command:** USE databaseName;
- **Purpose:** Switches to the specified database.

Example:

```
USE RetailStore;
```

Creating Tables

- **Command:** CREATE TABLE tableName (...);
- **Purpose:** Defines the structure of a table with columns and data types.

Key Concepts:

- **INT IDENTITY(1,1):** Creates an auto-incrementing integer column.
- **VARCHAR(size):** Defines a variable-length string column.
- **DECIMAL(precision, scale):** Represents numeric values with fixed decimal places.
- **BIT:** Represents boolean values (0 or 1).

NOT NULL:

- Ensures that a column cannot contain NULL values.

Primary Key:

- **Definition:** A primary key is a unique identifier for each record in a table. It ensures that no two rows have the same value for the primary key column.

- **Usage:** Designates a column (or a combination of columns) as the unique identifier.

Foreign Key:

- **Definition:** A foreign key is a column (or a set of columns) in one table that uniquely identifies a row of another table. It creates a relationship between the two tables.
- **Usage:** Enforces referential integrity between tables.

Example:

```
CREATE TABLE Sales (  
    saleId INT IDENTITY(1,1) PRIMARY KEY,          -- Primary Key  
    productName VARCHAR(255) NOT NULL,  
    saleDate DATE NOT NULL,  
    amount DECIMAL(10,2) NOT NULL,  
    isCompleted BIT NOT NULL,  
    customerId INT,                                  -- Foreign Key  
    FOREIGN KEY (customerId) REFERENCES Customers(customerId)  
);  
  
CREATE TABLE Customers (  
    customerId INT IDENTITY(1,1) PRIMARY KEY,      -- Primary Key  
    customerName VARCHAR(255) NOT NULL,  
    contactEmail VARCHAR(255) UNIQUE  
);
```

2.2 ALTER Statement:

The Alter Table command helps us to modify the structure of the table. If we need to add, delete or modify the columns in our existing table then we use the Alter Table Statement.

Syntax:

To **add column** to the existing table:

```
ALTER TABLE ExistingTableName  
ADD columnName datatype;
```

To **remove column** from the existing table:

```
ALTER TABLE ExistingTableName  
DROP columnName;
```

Lastly, to **modify the datatype of a column** in our existing table:

```
ALTER TABLE ExistingTableName  
ALTER COLUMN columnName newDataType;
```

Example:

```
ALTER TABLE Students ADD Email VARCHAR(100)  
ALTER TABLE Students ALTER COLUMN Email TEXT  
ALTER TABLE Students DROP COLUMN Email
```

2.3 DROP Statement:

As the name suggests, the DROP Statement is used to delete a table or a database.

- **Command:** DROP TABLE tableName;
- **Purpose:** Delete a table and its all data from the database.

```
DROP TABLE Students;
```

This will delete the **Students** table which we have created. There may be a situation when we just want to empty the table and not to delete the table itself. In such situations, we use the **TRUNCATE** table statement of SQL.

Suppose we have data of 50 old students in our table. But we don't need that data anymore. But we need the table as we need to keep a record of our students. In such situations, we will use the truncate table statement so that only the data gets deleted and the table is as it is.

Example:

```
TRUNCATE TABLE Students;
```

SQL Server Constraints:

Constraints in SQL Server are rules and restrictions applied on a column or a table such that unwanted data can't be inserted into tables. This ensures the accuracy and reliability of the data in the database.

We can create constraints on single or multiple columns of any table. Constraints maintain the data integrity and accuracy in the table.

Constraints can be classified into the following two types.

1. Column Type Constraints:

Definitions of these types of constraints is given when the table is created.

Syntax:

```
CREATE TABLE tableName (id int NOT NULL, salary int CHECK (salary > 500));
```

2. Table Type Constraints:

Definitions of these types of constraints is given after the creation of the table using the Alter Command.

Syntax:

```
ALTER TABLE tableName ALTER COLUMN columnName columnDataType  
CHECK(columnName > 600);
```

Moreover, SQL Server contains the following 6 types of constraints:

1. Not Null Constraint
2. Check Constraint
3. Default Constraint
4. Unique Constraint
5. Primary Constraint
6. Foreign Constraint

Let us understand each constraint briefly.

1. NOT NULL Constraint:

A Not null constraint restricts the insertion of null values into a column. If we are using a Not Null Constraint for a column then we cannot ignore the value of this column during an insert of data into the table.

Column Level:

Syntax:

```
CREATE TABLE tableName (columnName datatype CONSTRAINT, constraintName NOT NULL);
```

Example:

```
CREATE TABLE students (id int CONSTRAINT not_null_constraint NOT NULL);
```

Table Level:

Syntax:

```
ALTER TABLE tableName  
ALTER COLUMN columnName, columnDataType NOT NULL;
```

Example:

```
ALTER TABLE Students ALTER COLUMN id INT NOT NULL;
```

2. CHECK Constraint:

Limits the values that can be placed in a column based on a specified condition. This constraint ensures data meets certain criteria.

Syntax:

```
CREATE TABLE tableName (columnName datatype CHECK (condition));
```

Example:

```
CREATE TABLE Students (id INT, marks INT CHECK (marks >= 0 AND marks <= 100));
```

3. DEFAULT Constraint:

Sets a default value for a column if no value is provided during data insertion.

Syntax:

```
CREATE TABLE tableName (columnName datatype DEFAULT defaultValue);
```

Example:

```
CREATE TABLE Students (id INT, city VARCHAR(50) DEFAULT 'Unknown');
```

4. UNIQUE Constraint:

Ensures that all values in a column are unique, preventing duplicate values. A table can have multiple unique constraints, but each column under this constraint must hold unique data.

Syntax:

```
CREATE TABLE tableName (columnName datatype CONSTRAINT UNIQUE);
```

Example:

```
CREATE TABLE Students (id INT CONSTRAINT UNIQUE);
```

5. PRIMARY KEY Constraint:

Uniquely identifies each row in a table. A primary key column cannot contain NULL values, and each table can have only one primary key.

Syntax:

```
CREATE TABLE tableName (columnName datatype CONSTRAINT constraintName PRIMARY KEY);
```

Example:

```
CREATE TABLE Students (id INT CONSTRAINT pk_students_id PRIMARY KEY);
```

6. FOREIGN KEY Constraint:

Creates a link between two tables by establishing a relationship between a column in one table (the foreign key) and a primary key in another table. This enforces referential integrity.

Syntax:

```
CREATE TABLE tableName (columnName datatype, CONSTRAINT constraintName FOREIGN KEY (columnName) REFERENCES parentTable(parentColumn));
```

Example:

```
CREATE TABLE Enrollments (student_id INT, CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES Students(id));
```

* Provided a constraint name is **not** necessary as if left empty, the SQL Server will automatically assign a name.